

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Алгоритм Кнута-Морриса-Пратта

Студентка гр. 4345

Савонькина Е.Н.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм Кнут-Моррис-Пратта. Провести исследование его работы на различных размерах входных данных.

Основные теоретические положения.

Алгоритм КМП предназначен для поиска всех вхождений одной строки (образца) в другую строку (текст) за линейное время. Его ключевая идея состоит в том, чтобы при несовпадении символов не возвращаться назад по тексту, а использовать уже накопленную информацию о совпавших префиксах.

Ключевым компонентом алгоритма является префикс-функция π для строки s длины m , где $\pi[i]$ — длина наибольшего собственного префикса $s[0..i]$, который также является суффиксом $s[0..i]$. Собственный префикс — это любой префикс, кроме всей строки. Префикс-функция позволяет определить, сколько символов образца уже совпало, и на сколько можно сдвинуть образец при несовпадении.

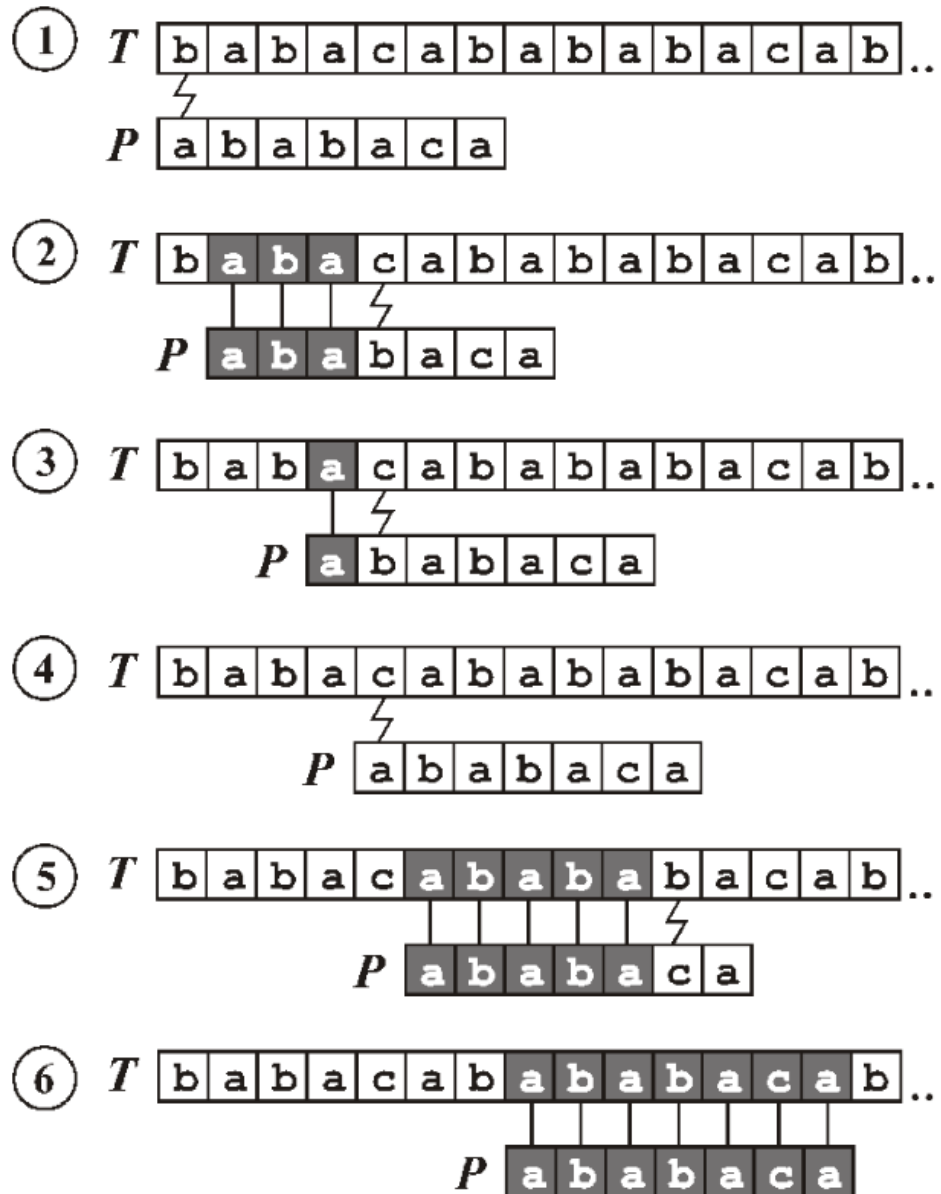
Пример работы префикс функции для образца «aasdfsaa»:

Шаг	Рассматриваемая часть образца	Максимальный собственный суффикс, равный префиксу	Длина
1	a	-	0
2	aa	a	1
3	aas	-	0
4	aasd	-	0
5	aasdf	-	0
6	aasdfs	-	0
7	aasdfs a	a	1
8	aasdfsaa	aa	2

Итоговая префикс функция: (0, 1, 0, 0, 0, 0, 1, 2)

Затем с её помощью осуществляется поиск: мы двигаемся по тексту и поддерживаем количество совпавших символов. При несовпадении не начинаем сравнение заново, а откатываемся к значению префикс-функции.

Пример поиска образца «ababаса» в тексте «babасababасab...»:



Задание 1.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P
- Вторая строка — T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Sample Input:

ab

abab

Sample Output:

0,2

Задание 2.

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A

- Вторая строка — B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Sample Input:

defabc

abcdef

Sample Output:

3

Реализация алгоритма.

Сначала программа реализует префикс-функцию. Для заданной строки она последовательно вычисляет для каждой позиции длину наибольшего префикса, который совпадает с суффиксом подстроки, заканчивающейся в этой позиции. Это делается с помощью переменной j , которая хранит текущую длину совпадения. Если символы не совпадают, происходит переход к более

короткому префиксу по уже вычисленным значениям. Таким образом, каждая позиция обрабатывается за константное время, и вся функция работает за линейное время.

Далее используется сам алгоритм Кнута–Морриса–Пратта. Он принимает образец и текст. Сначала строится префикс-функция для образца. Затем происходит проход по тексту: символы текста сравниваются с символами образца. Если символы совпадают, увеличивается длина текущего совпадения. Если нет — используется префикс-функция, чтобы определить, с какого места продолжить сравнение, не возвращаясь назад по тексту. Когда длина совпадения становится равной длине образца, найдено вхождение, и фиксируется его позиция.

В стандартной версии программы результатом является либо список всех вхождений образца в текст, либо первое найденное вхождение.

Для решения задачи циклического сдвига алгоритм применяется особым образом. Сначала считываются строки А и В. Проверяется, равны ли их длины, так как строки разной длины не могут быть циклическими сдвигами друг друга.

Далее используется ключевое свойство: если строка А является циклическим сдвигом строки В, то она обязательно будет подстрокой строки, полученной из В путем “удвоения”. Это означает, что достаточно проверить, входит ли А в последовательность символов строки В, пройденную дважды подряд.

Практически это реализуется без явного создания удвоенной строки. Вместо этого алгоритм КМП проходит по строке В два раза, используя индекс по модулю длины строки. Это позволяет избежать лишнего потребления памяти.

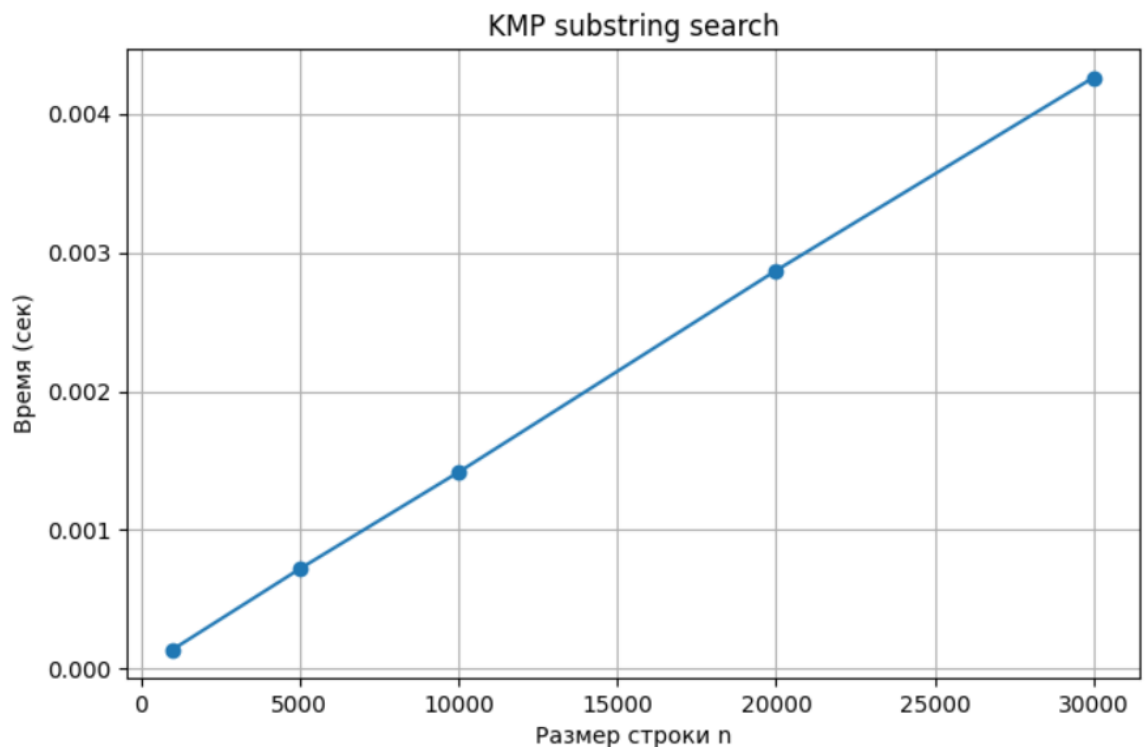
Оценка сложности.

Построение префикс-функции для строки длины n выполняется за линейное время $O(n)$. Это достигается за счёт того, что указатель j (длина текущего совпадения) может увеличиваться и уменьшаться, но суммарно за

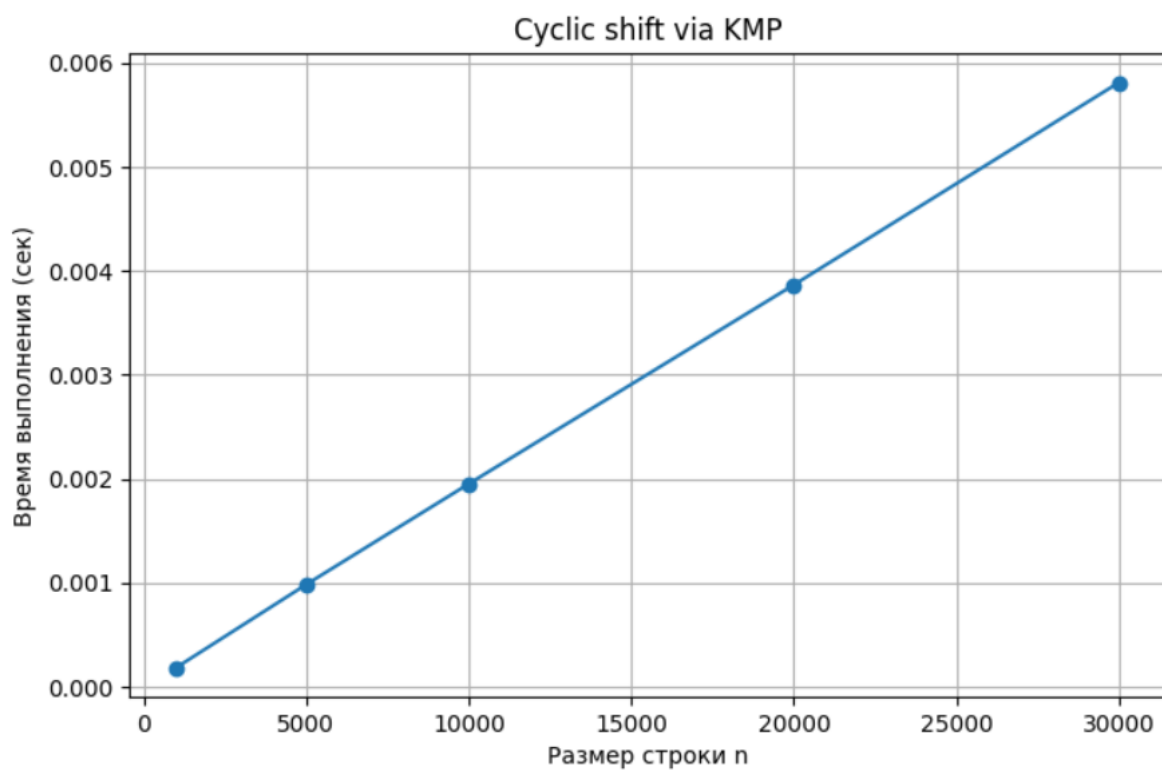
весь алгоритм он изменяется не более чем на n шагов вперёд и назад. Таким образом, вложенный цикл `while` в сумме не даёт квадратичной сложности.

Поиск подстроки в тексте длины m также выполняется за $O(m)$. Аналогично, несмотря на наличие вложенного цикла `while`, суммарное число переходов по префикс-функции ограничено длиной строки. Каждый символ текста обрабатывается константное число раз.

Итоговая временная сложность КМП составляет $O(n + m)$, где n — длина образца, m — длина текста. Память требуется $O(n)$ для хранения префикс-функции.



Для задачи циклического сдвига обе строки имеют длину n . Сначала проверка длины выполняется за $O(1)$. Затем строится префикс-функция для строки A за $O(n)$. После этого выполняется поиск строки A в «удвоенной» строке B .



Тестирование.

Таблица тестирования для задачи 1:

Строки	Результат	Комментарии
а	-1	Результат корректен.
а	-1	Результат корректен.
aaa aaa	0	Результат корректен.
а aaa	0,1,2	Результат корректен.
aaa а	-1	Результат корректен.

Таблица тестирования для задачи 2:

Строки	Результат	Комментарии
ab ba	1	Результат корректен.
ababa babab	-1	Результат корректен.
а aaa	-1	Результат корректен.

Вывод.

В ходе работы был изучен алгоритм КМП и его применение для поиска подстроки. На его основе реализовано эффективное решение задачи проверки циклического сдвига строк с линейной сложностью по времени и памяти.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Задание 1.

```
def prefix_function(P):
    print("Строим префикс функцию")
    n = len(P)
    pi = n * [0]
    j = 0

    for i in range(1, n):
        while j > 0 and P[i] != P[j]:
            j = pi[j - 1]

        if P[i] == P[j]:
            j += 1

        pi[i] = j
        print(f"Для pi[{i}] = {j}: "
              f"рассматриваем '{P[:i+1]}', "
              f"совпадающий префикс '{P[:j]}'" )

    print(f"Функция: {pi}")
    return pi

def kmp(P, T):
    pi = prefix_function(P)

    res = []
    j = 0

    print(f"\nИщем образец '{P}' в тексте '{T}'")

    for i in range(len(T)):

        print(f"\nРассматриваем символ текста '{T[i]}' (позиция {i})")

        while j > 0 and T[i] != P[j]:
            print(f"Не совпадает с образцом: '{T[i]}' != '{P[j]}'")
            print(f"Смотрим префикс-функцию pi[{j-1}] = {pi[j-1]}")
            j = pi[j - 1]
            print(f"Переходим к позиции в образце j = {j}")
```

```

        if T[i] == P[j]:
            print(f"Совпало: '{T[i]}' == '{P[j]}'")
            j += 1
            print(f"Уже: {P[:j]}")

        if j == len(P):
            start = i - j + 1
            print(f"\nсовпадение с позиции {start} в тексте")
            res.append(start)

            print(f"Используем префикс-функцию pi[{j-1}] = {pi[j-1]} для
продолжения поиска")
            j = pi[j - 1]

    print(f"\nИтоговые позиции: {res}")
    return res

if __name__ == '__main__':
    P = input()
    T = input()

    res = kmp(P, T)

    if res == []:
        print(-1)
    else:
        print(','.join(map(str, res)))

```

Задание 2.

```

def prefix_function(P):
    n = len(P)
    pi = [0] * n
    j = 0

    print("Строим префикс функцию")

    for i in range(1, n):
        while j > 0 and P[i] != P[j]:
            j = pi[j - 1]

        if P[i] == P[j]:
            j += 1

```

```

        pi[i] = j
        print(f"Для pi[{i}] = {j}: рассматриваем '{P[:i+1]}', совпадающий
префикс '{P[:j]}')

    print(f"Функция: {pi}")
    return pi

def kmp_cyclic(A, B):
    n = len(A)
    pi = prefix_function(A)
    j = 0

    print(f"\nИщем циклический сдвиг '{A}' в '{B*2}'")

    for i in range(2 * n):

        c = B[i % n]

        print(f"\nРассматриваем символ текста '{c}' (позиция {i % n} в B, шаг
{i})")

        while j > 0 and c != A[j]:
            print(f"Не совпадает с образцом: '{c}' != '{A[j]}'")
            print(f"Смотрим префикс-функцию pi[{j-1}] = {pi[j-1]}")
            j = pi[j - 1]
            print(f"Переходим к позиции в образце j = {j}")

        if c == A[j]:
            print(f"Совпало: '{c}' == '{A[j]}'")
            j += 1
            print(f"Уже: {A[:j]}")

        if j == n:
            pos = i - n + 1
            print(f"\нсовпадение с позиции {pos} в удвоенной строке")

            if pos < n:
                shift = (n - pos) % n
                print(f"циклический сдвиг = {shift}")
                return shift

```

```

        else:
            print("совпадение вне допустимого диапазона")
            return -1

print("\nсовпадение не найдено")
return -1

if __name__ == '__main__':
    A = input().strip()
    B = input().strip()

    if len(A) != len(B):
        print(-1)
    else:
        print(kmp_cyclic(A, B))

```